

Build Your First Parameterized dbt Pipeline

Real-World Context

You're a data engineer at a retail company that processes daily sales transactions. Currently, your team manually updates SQL queries each day to process the previous day's sales data, spending 30 minutes every morning updating date filters, regional parameters, and status criteria. This manual process is error-prone and prevents automated scheduling. Your task is to create a parameterized dbt model that can process any date range automatically based on execution parameters.

Learning Focus

You will demonstrate these capabilities:



Design parameterized SQL queries that adapt to different execution contexts

Apply dbt variable syntax to create reusable data transformation logic

Prerequisites & Setup

Access to dbt CLI or dbt Cloud environment

Sample retail sales dataset (or use provided sample data structure)

Basic understanding of SQL SELECT statements

Text editor for SQL development

Sample Data Structure

orders table

order_id, customer_id, order_date, total_amount, status, region,
product_category

Your Challenge

Transform this static sales analysis query into a parameterized dbt model that can automatically process any date range and region combination

SELECT

order_date,

region,

COUNT(*) as order_count,

SUM(total_amount) as total_sales,

AVG(total_amount) as avg_order_value

Transform this static sales analysis query into a parameterized dbt model that can automatically process any date range and region combination

FROM orders

WHERE order_date = '2024-01-15'

AND status = 'completed'

AND region = 'North'

GROUP BY order_date, region

Scaffolded Instructions

Step 1: Set Up Your dbt Model Structure:

- Create a new dbt model file named `daily_sales_summary.sql`
- Define the basic `SELECT` structure without parameters initially
- Test that your model runs with hardcoded values

Step 2: Add Variable Configuration:

- Create or update your `dbt_project.yml` file to include variables for:
- `analysis_date` (default to current date)
- `target_region` (default to 'All')
- `order_status` (default to 'completed')
- Test your configuration by running `dbt compile` to verify syntax

Step 3: Implement Parameter Substitution:

- Replace hardcoded date with `{{ var('analysis_date') }}`
- Replace hardcoded region with conditional logic that handles 'All' vs specific regions
- Replace hardcoded status with `{{ var('order_status') }}`
- Add parameter validation using `{% if %}` statements

Step 4: Test Multiple Parameter Combinations:

- Run your model with default parameters
- Test with a specific region: `--vars '{"target_region": "South"}'`
- Test with different status: `--vars '{"order_status": "pending"}'`
- Verify results change appropriately with parameter modifications

Step 5: Add Business Logic Enhancement:

- Include conditional aggregations that only calculate certain metrics when specific regions are selected
- Add date range expansion capability (process multiple days based on parameter)
- Document your parameters and their expected values in the model file

Success Indicators

Your dbt model compiles and runs successfully with default parameters

Model produces different results when run with different parameter values

SQL code includes proper parameter substitution using `{{ var() }}` syntax

Model includes conditional logic that handles different business scenarios

Documentation comments explain parameter usage and expected values

Common Pitfalls & Prevention

Parameter syntax errors Remember dbt uses `{{ var('param_name') }}` not `${param_name}` test compilation frequently

Missing default values Always provide defaults in `dbt_project.yml` to prevent runtime errors when parameters aren't specified

SQL injection risks Use dbt's built-in parameter validation rather than direct string concatenation for dynamic queries